

```
newcabinet = insert_cabinet(cabinet,5)
print(newcabinet)
```

Listing 4-1: *Inserting a numbered file into our cabinet*

When you run the code in [Listing 4-1](#), it will print out `newcabinet`, which you can see includes our new “file,” 5, inserted into our cabinet at the correct location (between 4 and 6).

It’s worthwhile to think for a moment about one edge case of insertion: inserting into an empty list. Our insertion algorithm mentioned “proceeding sequentially through every file in the filing cabinet.” If there are no files in the filing cabinet, then there is nothing to proceed through sequentially. In this case, we need to heed only the last sentence, which tells us to insert our new file at the beginning of the cabinet. Of course, this is easier done than said, because the beginning of an empty cabinet is also the end and the middle of the cabinet. So all we need to do in this case is insert the file into the cabinet without regard to position. We can do this by using the `insert()` function in Python and inserting at location 0.

Sorting via Insertion

Now that we’ve rigorously defined insertion and know how to perform it, we’re almost at the point where we can perform an insertion sort. Insertion sort is simple: it takes each element of an unsorted list one at a time and uses our insertion algorithm to insert it correctly into a new, sorted list. In filing cabinet terms, we start with an unsorted filing cabinet, which we’ll call “old cabinet,” and an empty cabinet, which we’ll call “new cabinet.” We remove the first element of our old unsorted cabinet and add it to our new empty cabinet, using the insertion algorithm. We do the same with the second element of the old cabinet, then the third, and so on until we have inserted every element of the old cabinet into the new cabinet. Then, we forget about the old cabinet and use only our new, sorted cabinet. Since we’ve been inserting using our insertion algorithm, and it always returns a sorted list, we know that our new cabinet will be sorted at the end of the process.

In Python, we start with an unsorted cabinet and an empty `newcabinet`: